



DIPARTIMENTO DI INGEGNERIA ELETTRICA
E TECNOLOGIE DELL'INFORMAZIONE

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

FIELD AND SERVICE ROBOTICS

FROM STANDING STILL TO BOUNDING: GAIT LEARNING ON ANYmal-C

Prof.
Fabio Ruggiero

Studenti
William Notaro P38/271
Salvatore Piccolo P38/256

Anno Accademico 2024/2025

Contents

1	Introduction	2
1.1	Motivation	2
1.2	A Journey of obstacles: Self-Congratulations	2
1.3	Structure of the report	4
2	SEA Model Learning	5
2.1	Serial Elastic Actuators	5
2.1.1	Physical Platform: ANYmal-C Robot	5
2.2	Actuator Network	6
2.2.1	ActNet Architecture	6
2.2.2	Training Pipeline	7
2.2.3	Simulation deployment	8
3	First Experiments	10
3.1	Analysis of Simulation Data	10
3.2	Reward function	11
3.3	Simulation with constant velocity command	13
3.4	Simulation with added curriculum	16
4	Inducing a Bounding Gait	18
4.1	Reward Shaping for Bounding Behavior	19
4.2	Simulations and Results	20
5	Conclusions	23

Chapter 1

Introduction

The field of legged robotics continues to push the frontier of what autonomous machines can achieve in terms of mobility, adaptability, and dynamic interaction with complex terrains. Among quadrupedal robots, ANYmal-C stands out as a highly capable platform, capable of navigating challenging environments through dynamic locomotion strategies. Traditionally, such behaviors have been engineered through model-based control techniques, requiring careful tuning and detailed understanding of system dynamics. However, recent advances in machine learning have introduced new paradigms where control policies can be learned directly from data using reinforcement learning (RL), enabling the robot to acquire complex motor behaviors from interaction with its environment.

This work presents the development and training of a reinforcement learning agent to teach ANYmal-C a dynamic and non-trivial locomotion pattern: the bound gait. Unlike walking or trotting, bounding requires precise timing, contact coordination, and high-speed maneuvers. This report documents the methodological steps, challenges, and design choices made using Isaac Lab, NVIDIA’s research framework built on Isaac Sim, to simulate, control, and iteratively refine the learned policy for ANYmal-C. The overall simulation structure that will be utilized over all of the project is shown in Fig. 1.1.

1.1 Motivation

This project originates as a natural continuation and deepening of Homework 4, which focused on legged robotics. While the homework provided valuable insights into locomotion strategies and control objectives, it lacked actuator modeling, making it impossible to conduct energy consumption analyses.

To fill this gap, we decided to dive into a more realistic simulation framework and develop a reinforcement learning-based control strategy. Our goal was to experience first-hand the development process of a data-driven controller via reward shaping and curriculum design.

1.2 A Journey of obstacles: Self-Congratulations

Completing this project was not merely a technical endeavor—it was a continuous exercise in resilience and ingenuity. A key limitation from the beginning was that the hardware used for development did not meet the official minimum specifications required to run Isaac Sim.

The following table outlines the hardware specifications of the system we used for our simulations, alongside the recommended system requirements for running IsaacLab 4.5.0.

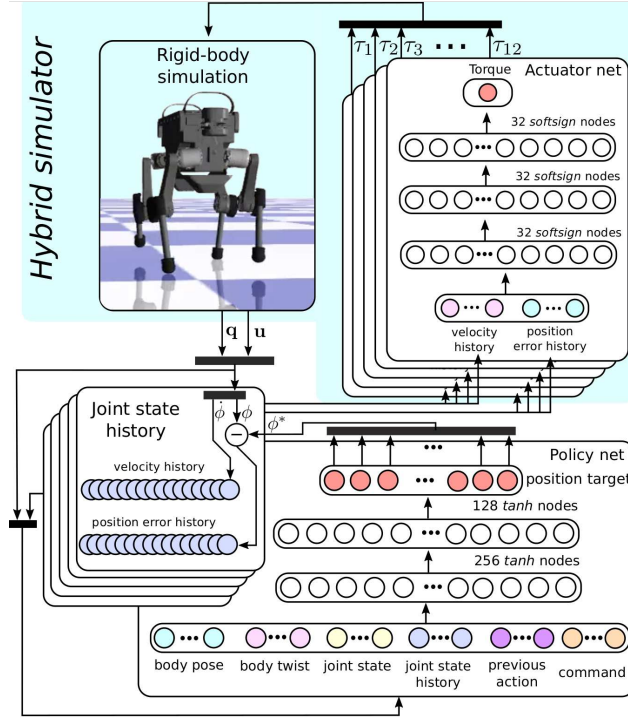


Figure 1.1: **Overall Simulation Structure.** The policy network maps the current observation and the joint state history to the joint position targets. The actuator network maps the joint state history to the joint torque, which is used in rigid-body simulation. The state of the robot consists of the generalized coordinate q and the generalized velocity u . The state of a joint consists of the joint velocity $\dot{\phi}$ and the joint position error, which is the current position ϕ subtracted from the joint position target ϕ^* .

Our System Specifications	IsaacLab 4.5.0 Requirements
CPU: Gen Intel Core i5 (11th Gen)	CPU (Minimum): Intel Core i7 (7th Gen) / AMD Ryzen 5 CPU (Recommended): Intel Core i7 (9th Gen) / AMD Ryzen 7 CPU (Ideal): Intel Core i9, X-series or higher / AMD Ryzen 9, Threadripper or higher
RAM: 16GB	RAM (Minimum): 32GB (More RAM is recommended for advanced use cases and training) RAM (Recommended/Ideal): 64GB
GPU: GeForce RTX 3060 Laptop	GPU (Minimum): GeForce RTX 3070 GPU (Recommended): GeForce RTX 4080 GPU (Ideal): RTX Ada 6000
VRAM: 6GB	VRAM (Minimum): 10GB VRAM (Recommended): 16GB VRAM (Ideal): 48GB

Despite this, we managed to complete the full training and evaluation pipeline for a high-dimensional quadruped robot. The road was paved with many challenges, including:

- **Out-of-memory crashes during environment initialization:** We addressed this by drastically reducing the number of parallel environments (`--num_envs`).

- **PhysX GPU kernel failures:** A crucial optimization involved offloading physical calculations to the CPU instead of the GPU, particularly when simulating on complex or rough terrains. This approach allowed for the rendering of intricate environments and the inclusion of more simulated units.
- **Headless Mode Training:** For training purposes, a maximum of 2048 trainable robots per simulation was achievable by utilizing the `--headless` flag. This allowed training to proceed efficiently without requiring visual feedback, thereby conserving computational resources.
- **Visualized Robots during Simulation:** During visualization of simulations, only a single trained robot could initially be displayed on flat terrain, and none in rough terrains. However, by using the `--cpu` flag, we were able to render multiple robots.

1.3 Structure of the report

In the following, the report is organized into four main chapters, each addressing a different aspect of the project and following the natural progression from motivation to implementation, experimentation, and conclusions.

- Chapter 2 introduces the Serial Elastic Actuators (SEAs) used in the ANYmal-C robot and explains their complex dynamics. It presents the supervised learning approach adopted to model actuator behavior and details the training pipeline: from raw data collection to the creation of the ActuatorNet and its deployment in IsaacLab. Special attention is given to network design, data preprocessing, and compatibility issues.
- Chapter 3 describes the experiments conducted to evaluate the learned control policy. Several evaluation setups are compared, including constant command inputs and curriculum-based velocity tracking.
- Chapter 4 focuses on achieving a biologically-inspired bounding gait in ANYmal-C. It elaborates on the gait-specific reward shaping strategies, including front-hind synchronization, flight phase encouragement, and symmetry. The chapter ends with the analysis of the outcomes of the final training stage along with a discussion on the robot autonomy in bound mode.
- Chapter 5 summarizes the main results and contributions of the work. It reflects on the effectiveness of the approaches used and outlines potential directions for future improvements, such as smooth gait transitions, gait diversity, and sim-to-real transfer.

Chapter 2

SEA Model Learning

2.1 Serial Elastic Actuators

Actuators play a fundamental role in the operation of legged robotic systems. Most commonly, these robots are equipped with hydraulic actuators or electric motors paired with gearboxes. Despite their widespread use, modeling these actuators analytically presents significant challenges, since their dynamics are inherently nonlinear. Furthermore, they are governed by cascaded feedback loops and include multiple internal states—many of which are not directly observable. This complexity is further exacerbated by the limited documentation provided by manufacturers, making it often infeasible to build a reliable analytical model.

To address this, the authors in [1] adopt a data-driven approach based on supervised learning to learn a mapping from control inputs to torques. More precisely, they train a neural network—referred to as an actuator network (or, ActNet)—that estimates the joint torques based on a history of joint position errors (i.e., the difference between commanded and actual joint positions) and joint velocities. In addition, they assume actuator dynamics are decoupled, allowing them to train one independent model for all the actuators.

It is also important to note that actuator dynamics are only partially observable. Therefore, the model must implicitly infer these hidden states from the observable quantities.

2.1.1 Physical Platform: ANYmal-C Robot

We use the ANYmal-C quadruped robot to validate the proposed actuator modeling approach. ANYmal-C is a medium-sized, dog-like robot weighing approximately 32 kg. Each of its four legs includes three actuated joints - Hip Abduction/Adduction (HAA), Hip Flexion/Extension (HFE), and Knee Flexion/Extension (KFE) - making a total of 12 degrees of freedom. Its Universal Scene Description (USD) structure and its appearance in the simulation framework is shown in Fig. 2.1.

Each joint is equipped with a Series Elastic Actuator (SEA), Fig. 2.2, consisting of an electric motor, a high-reduction gearbox, an elastic spring element, and two encoders to measure spring deflection and joint output. Some of the advantages of the SEA are that the applied forces can be precisely controlled and the shock tolerance and compliance with the environment can be increased.

A PD controller operates at the joint level to generate torque commands. However, the actual torque generation involves a complex cascade of nested control loops. First, a PD controller converts the joint position command into a desired torque. Then, a motor-level PID controller translates this into a desired current. This current is then fed into a Field-Oriented Controller (FOC), which produces the required phase voltage for the motor. The generated torque passes through the transmission and finally through the elastic element, where the spring deflection produces the torque at the

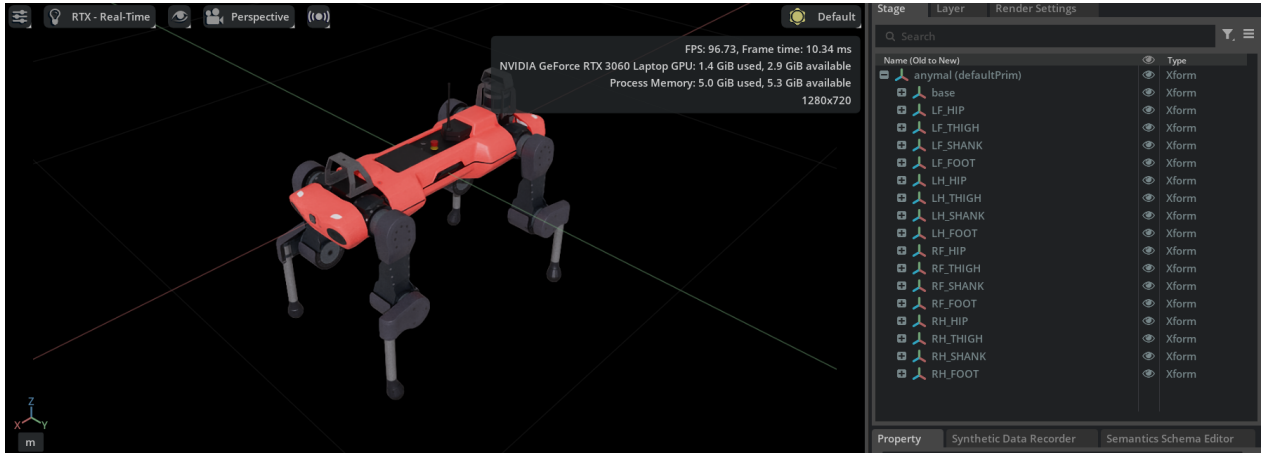


Figure 2.1: Appearance and structure of the ANYmal-C USD file inside the IsaacSim simulation framework. The robot is composed of a central body (base) and four legs, each equipped with three actuated joints.

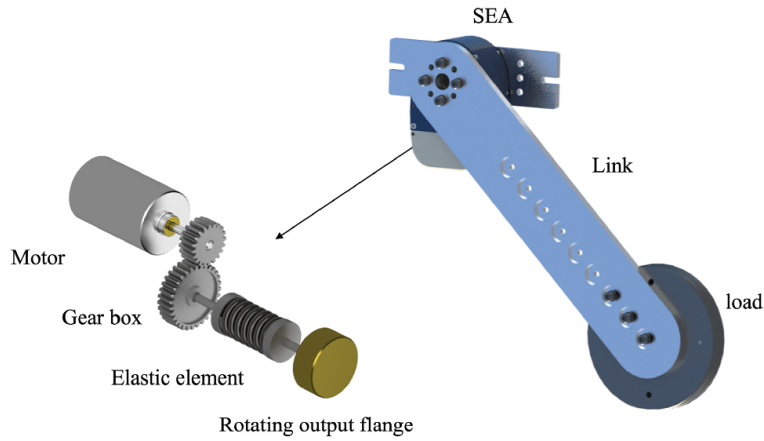


Figure 2.2: A single-link SEA system.

joint. These layers of nested dynamics introduce hidden internal states, making the overall system difficult to model and control using traditional approaches—further justifying the use of learning-based methods.

2.2 Actuator Network

2.2.1 ActNet Architecture

The goal of the ActNet is to estimate actuator torques while internally compensating for the unobservable dynamics, based only on joint position error and velocity histories. The actuator model is a lightweight multi-layer perceptron (MLP) with the following architecture:

- An input layer of size $2N$ (N position errors + N velocities);
- Three hidden layers with 32 SoftSign-activated units each;
- A single output neuron for the predicted torque.

The architecture can be visualized in the top right of Fig. 1.1. Each input sample to the network includes a short time window: the current state t and two past states corresponding to $t-0.01$ and $t-0.02$ seconds (i.e., $N = 3$). This temporal input window is a trade-off: too sparse a history fails to capture fast actuator dynamics (particularly above 100 Hz), while too dense a history increases the risk of overfitting and computational burden. In practice, the authors in [1] tuned this

hyperparameter based on the validation error. Thanks to the small size of the network, this tuning process is relatively fast.

2.2.2 Training Pipeline

To learn a model of the ANYmal-C actuators, we adopted a multi-stage training pipeline that combines simulation-based logging, external preprocessing, supervised learning, and model deployment. The following steps outline the complete actuator training workflow:

1. Raw Data Collection from Real World Robot

The authors in [1] execute a scripted movement routine for the real world ANYmal robot. During these scripted runs, among other quantities, joint position errors (the difference between commanded and measured joint angles), joint velocities, and applied joint torques are logged at high frequency (400 Hz). The logs are saved into .silo files that are provided by the authors at their Git repository [2]. In other words, .silo files contain the raw data that are logged from the real world robot and constitute the starting point for the next stage.

2. Data pre-processing in MATLAB and Dataset Generation

The logged raw data, before being fed to the training process, must be processed, and the same authors also provide MATLAB scripts capable of reading the .silo files. These scripts preprocess the raw signals by handling missing entries, aligning the time series, and extracting histories of relevant quantities. Finally, they compile the processed data into binary datasets that can be used for training. An example dataset contains 2273 samples, where each sample consists of time histories of position error and velocity, along with the corresponding ground truth torque label. The number of elements of the time history depends on the frequency on which the network must be trained and on how many past samples one wants to include. For example, if one trains the ActNet at 100Hz and chooses to include also two past values, then an input sample contains 6 scalars (3 for position error and 3 for velocities). The main MATLAB script used to complete this stage is `processData.m`.

3. Dataset Scaling

Authors in [1], however, do not provide scripts for the network training. Therefore, we had to develop our own training process. After loading the binary data, and before training the neural network, a further data processing has been required, i.e. the inputs were normalized using a `StandardScaler` to improve training stability. In general, a scaler is a preprocessing tool that transforms the features to a normalized space. A `StandardScaler` standardizes features of the dataset by removing the mean and scaling to unit variance:

$$x' = \frac{x - \mu}{\sigma} \quad (2.1)$$

4. Supervised Learning Using Keras

From the obtained dataset, 90% of the data was used for training while the remaining 10% was reserved for validation. The training was performed using the Keras framework. In particular, it was performed using the RMSProp optimizer and mean squared error loss. The model quickly converged, achieving low validation error thanks to the small size of the network and the richness of the dataset, as shown in Fig. 2.3.

In addition, from Fig. 2.4 we can conclude that the trained network nearly perfectly predicts the torque from the validation data. For completing this stage, `act_net_training.py` has been used for model training and `act_net_test.py` for model evaluation.

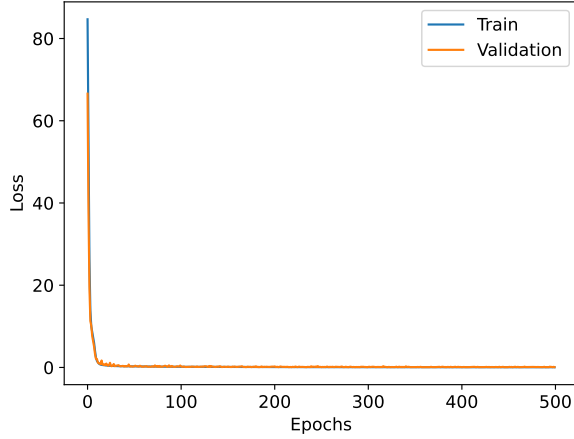


Figure 2.3: Training loss and validation loss curves for the ActNet_400Hz model.

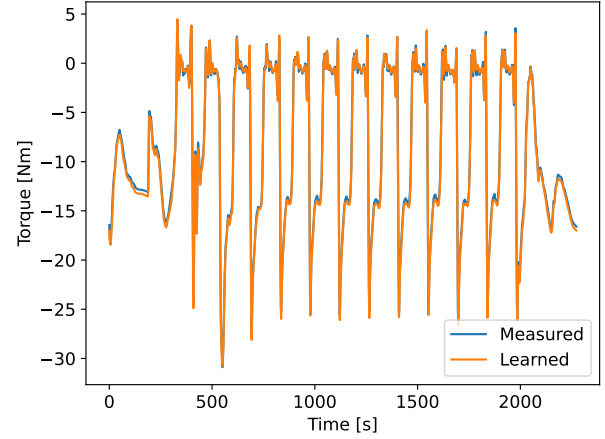


Figure 2.4: Torque prediction results for the ActNet_400Hz model.

2.2.3 Simulation deployment

To deploy the learned model inside IsaacLab (which uses PyTorch), we converted the trained Keras¹ model to a TorchScript module, which is an optimized format of a PyTorch model. The conversion process involved:

1. Converting the trained Keras model into an intermediate standardized format, called ONNX (Open Neural Network Exchange), using the `tf2onnx` library.
2. Converting the ONNX model into a PyTorch model via `onnx2pytorch` library.
3. Tracing the PyTorch model using `torch.jit.trace` and exporting it in a TorchScript (.pt) format. The tracing method is interesting in that it allows to infer the architecture of a neural network by 'tracing' (i.e., following) a dummy input that flows from the input layer to the output layer.

The conversion process of the TensorFlow model to the compiled TorchScript model is carried by `tensorflow2torchscript.py`.

One of the trained models we experimented with is ActNet_100Hz.pt, which is a direct conversion of the actuator network used by the original authors. This model was successfully integrated into the IsaacLab framework, meaning that it was properly loaded and executed within the simulation environment. However, when deployed on ANYmal, the behavior it induced was clearly undesirable: the robot exhibited instability, collapsing shortly after the simulation began and performing erratic movements (Fig. 2.5). We hypothesize two main causes behind this failure:

1. First, there may be a mismatch in scaling conventions between the original data and the IsaacLab API, particularly regarding how the measured joint states (e.g., position errors and velocities) are normalized and passed to the actuator model.
2. Second, the network itself may lack generalization capability across joints. Although all joints use the same actuator architecture, the network was trained solely on data from a single joint—a limitation stemming from the MATLAB scripts provided by the authors, which only generate training data for one joint. Furthermore, we found the MATLAB pipeline somewhat unsatisfying from a machine learning standpoint: the input dataset includes torque values,

¹The ActNet was initially trained in Keras and not directly in PyTorch since, during the project development, we followed a 3-day seminar dedicated to AI models training in Keras. Only when trying to deploy the model in IsaacLab we discovered that a conversion to TorchScript was needed.

which ideally should be reserved for the target set, not the input features.

Given these issues, and considering that our project is not centered on machine learning techniques, we chose not to pursue further debugging or retraining of `ActNet_100Hz.pt`. Instead, we reverted to using the original LSTM-based actuator model already integrated in IsaacLab, which offered robust and stable performance. Therefore, the results and analyses presented in the remainder of this report are based on the default actuator model provided by the framework.

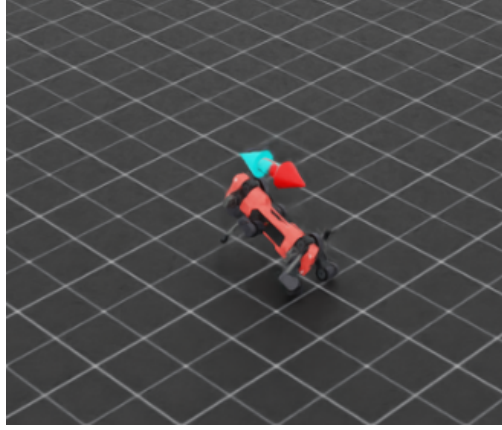


Figure 2.5: Deployment failure of the custom `ActNet_100Hz.pt` actuator model. Despite successful loading into the IsaacLab simulation, the robot collapses due to likely issues in input scaling or poor generalization of the network trained on data from a single joint.

Chapter 3

First Experiments

3.1 Analysis of Simulation Data

The empirical validation of our Reinforcement Learning (RL) agent is performed by analyzing time-series data extracted from simulations in Isaac Lab. This data, encapsulated in a `.mat` file, provides a comprehensive snapshot of the robot’s state and its interactions with the environment. The following subsections detail the key variables used in our analysis, their definitions, and the rationale for their use as implemented in our MATLAB post-processing scripts.

joint_vel **Definition:** `joint_vel` represents the angular velocities of all joints. **Purpose:** In our analysis, we extract these velocities in order to perform a power analysis on each robot’s leg to later define the battery autonomy.

applied_torque **Definition:** `applied_torque` is the final torque applied to the joints after accounting for limitations such as maximum effort (clipping). **Purpose:** We extract the applied torques to assess the mechanical power of each leg to conduct a power analysis later in the report.

commands **Definition:** These are the high-level velocity commands issued to the robot, comprising the desired linear (v_x and v_y) and angular velocity (ω_z) of the robot with respect to the world frame expressed in the robot’s base frame. **Purpose:** We plot the commands to correlate the robot’s observed behavior with the intended task. This helps verify that the policy is correctly interpreting and responding to high-level objectives.

root_link_state_w **Definition:** These variables represent the state of the robot’s base in the world frame ('w'). The state is a vector containing position, orientation (as a quaternion), linear velocity, and angular velocity. **Purpose:** Extracting the root link states is fundamental for analyzing the robot’s overall ability to follow the velocity commands.

contact_forces **Definition:** This 3D tensor contains the reaction forces (F_x, F_y, F_z) measured at various contact sensors on the robot’s body, most importantly the feet. **Purpose:** These forces are paramount for understanding the robot’s interaction with the ground. In our script, we specifically use the vertical force component (F_z) to determine the exact moments of ground contact for each foot. By applying a threshold, we generate a **Gait Diagram**, a crucial visualization that illustrates the robot’s walking pattern (e.g., trot, bound) and timing.

3.2 Reward function

We now describe the reward function used in the **AnymalCEnv** RL environment, built to train the ANYmal-C quadruped robot to perform stable and efficient locomotion.

The reward function is carefully crafted to promote desirable locomotion behavior, penalize instability and inefficiency, and ensure adherence to high-level commands (e.g., desired velocity and orientation).

The total reward R at each simulation step is defined as the sum of multiple components:

$$R = \sum_{i=1}^N r_i \quad (3.1)$$

Each r_i corresponds to a specific aspect of the robot's behavior. Rewards are weighted by scalar coefficients defined in the environment configuration file. Below, we describe each individual reward component, including its mathematical formulation, intuition, and intended behavioral outcome.

1. Linear Velocity Tracking (`track_lin_vel_xy_exp`)

$$r_{\text{lin_vel}} = \exp\left(-\frac{\|\mathbf{v}_{xy}^{\text{cmd}} - \mathbf{v}_{xy}^{\text{actual}}\|^2}{0.25}\right)$$

Encourages the robot to match the commanded linear velocity in the X and Y directions. Higher reward is given when the robot follows the command precisely.

2. Angular Velocity Tracking (`track_ang_vel_z_exp`)

$$r_{\text{yaw_rate}} = \exp\left(-\frac{(\omega_z^{\text{cmd}} - \omega_z^{\text{actual}})^2}{0.25}\right)$$

Aligns the robot's yaw rate with the desired angular command.

3. Vertical Velocity Penalty (`lin_vel_z_l2`)

$$r_{\text{z_vel}} = -(\dot{v}_z^{\text{actual}})^2$$

Penalizes vertical motion to keep the robot grounded. Discourages jumping or bouncing.

4. Angular Velocity XY Penalty (`ang_vel_xy_l2`)

$$r_{\text{ang_vel_xy}} = -\|\boldsymbol{\omega}_{xy}^{\text{actual}}\|^2$$

Penalizes undesired rotations around roll and pitch axes.

5. Joint Torque Penalty (`dof_torques_l2`)

$$r_{\text{torque}} = -\|\boldsymbol{\tau}\|^2$$

Reduces excessive actuation effort, thus encouraging energy-efficient locomotion.

6. Joint Acceleration Penalty (`dof_acc_l2`)

$$r_{\text{accel}} = -\|\ddot{\boldsymbol{\theta}}\|^2$$

Penalizes abrupt changes in joint motion, encourages smooth and natural movements.

7. Action Rate Penalty (action_rate_l2)

$$r_{\text{action_rate}} = -\|a_t - a_{t-1}\|^2$$

Discourages high-frequency control oscillations.

8. Feet Air Time Reward (feet_air_time)

$$r_{\text{air_time}} = \sum_{\text{feet}} (\text{air_time} - 0.5) \cdot \mathbf{1}_{\text{first_contact}} \cdot [\|v_{\text{cmd}}\| \geq 0.1]$$

where $[\cdot]$ is the indicator function that is 1 when the condition inside is true, and 0 otherwise. Encourages effective footstep cycles when locomotion is required. The reward is only applied when feet land on the ground.

9. Undesired Contacts Penalty (undesired_contacts)

$$r_{\text{undesired}} = - \sum_{\text{undesired_body_ids}} \mathbf{1}_{\text{contact} > 1.0}$$

Penalizes collisions with body parts not intended to touch the ground. Prevents falling, dragging, or crawling behavior.

10. Flat Orientation Penalty (flat_orientation_l2)

$$r_{\text{flat}} = -\|\mathbf{g}_{xy}^{\text{proj}}\|^2$$

Penalize body tilt by comparing gravity's projection on the robot's body frame. A perfectly upright body would have these values close to zero (since gravity should only act in Z). Encourages the robot to stay upright with a level torso.

Note: Reward coefficients (scaling factors) are configurable in the environment's configuration files and play a crucial role in shaping the final behavior. Table 3.1 shows the default scaling factors applied to each reward term.

Parameter	Value
lin_vel_reward_scale	1.0
yaw_rate_reward_scale	0.5
z_vel_reward_scale	-2.0
ang_vel_reward_scale	-0.05
joint_torque_reward_scale	-2.5e-5
joint_accel_reward_scale	-2.5e-7
action_rate_reward_scale	-0.01
feet_air_time_reward_scale	0.5
undesired_contact_reward_scale	-1.0
flat_orientation_reward_scale	-5.0

Table 3.1: Reward Scale Parameters

3.3 Simulation with constant velocity command

To evaluate the locomotion capabilities of the trained ANYmal C policy, we trained the robot to follow a target forward velocity uniformly varying between 0.0 m/s and 1.0 m/s. At the of the training, four test simulations were run. In each simulation a different constant velocity command was given to the robot. The velocities chosen for the tests are: 0.25, 0.5, 1.0, and 2.0 m/s. These simulations not only gave a clear overview of the robot capabilities, but, as it will be made clear, it underlined how a training curriculum is needed in order to achieve better generalization at tracking velocity commands.

The velocity tracking was evaluated by comparing the commanded velocities with the robot's root link frame's velocity. In Figs. 3.1-3.4 it can be noticed how the robot is able to fairly track the commanded velocities seen during training, while in the case of 2.0 m/s, a commanded velocity not seen during training, it does not follow the velocity command properly.

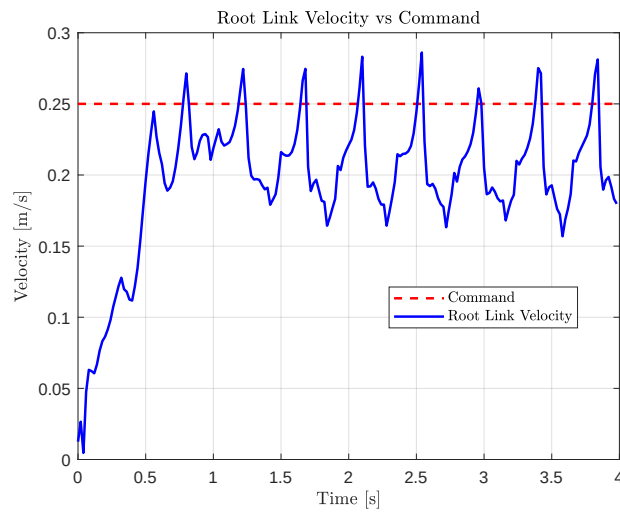


Figure 3.1: Velocity tracking analysis at command velocity 0.25 m/s.

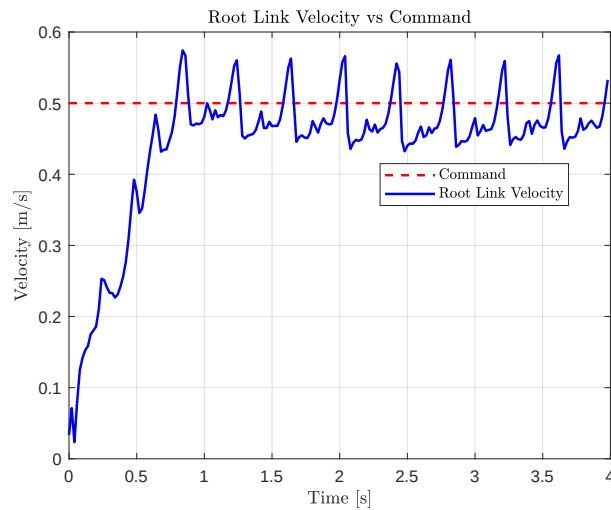


Figure 3.2: Velocity tracking analysis at command velocity 0.5 m/s.

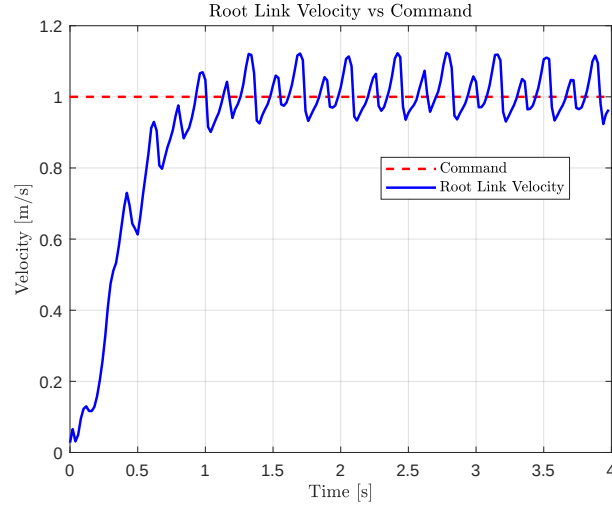


Figure 3.3: Velocity tracking analysis at command velocity 1.0 m/s.

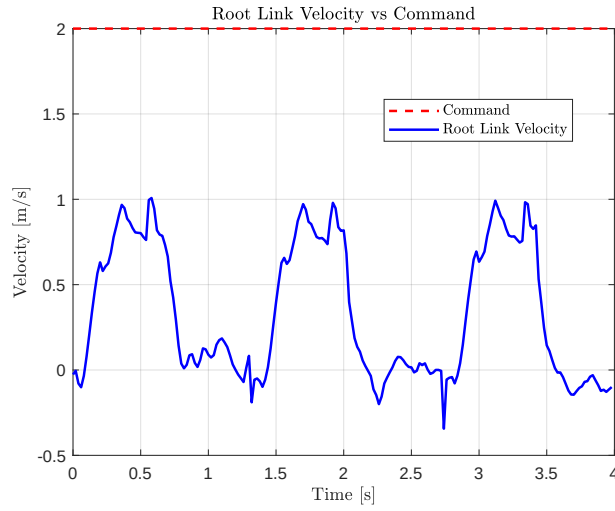


Figure 3.4: Velocity tracking analysis at command velocity 2.0 m/s.

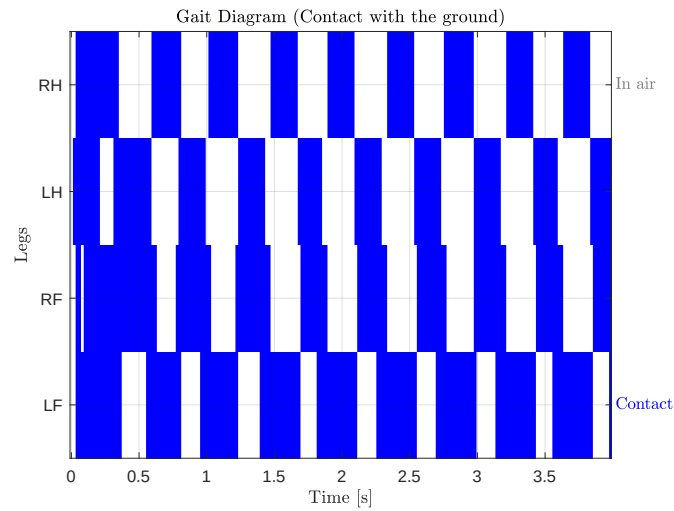


Figure 3.5: Gait analysis at command velocity 0.25 m/s. The robot follows the trajectory stably with consistent foot-ground contacts.

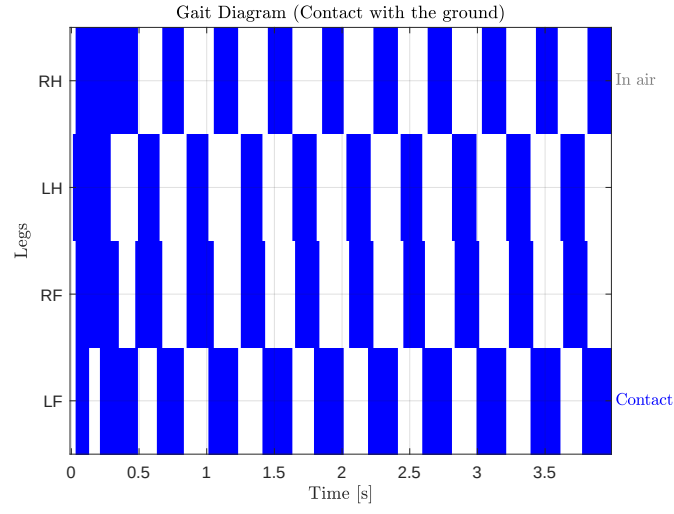


Figure 3.6: Gait analysis at command velocity 0.5 m/s. Stable locomotion and smooth transitions between stance and swing phases.

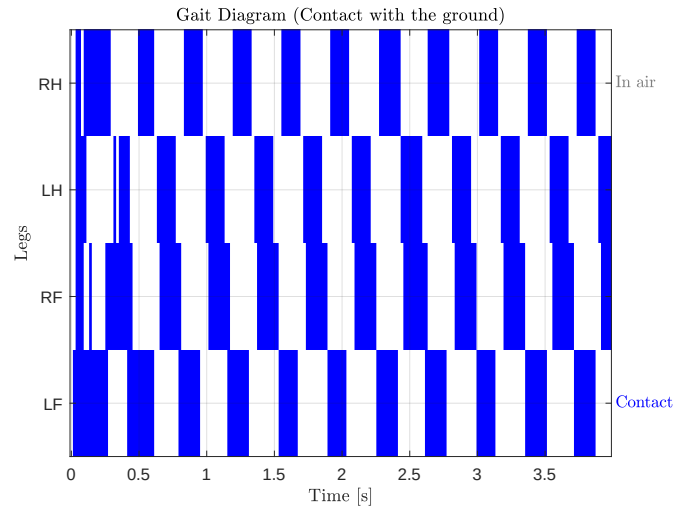


Figure 3.7: Gait analysis at command velocity 1.0 m/s. The robot maintains balance and successfully completes the trajectory.

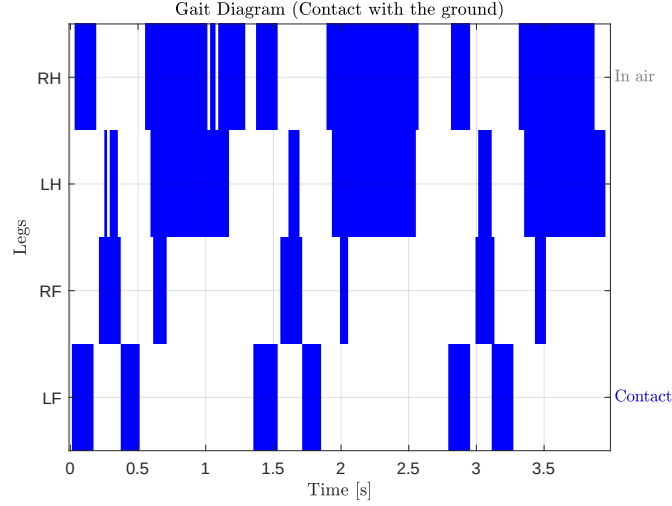


Figure 3.8: Gait analysis at command velocity 2.0 m/s. The robot loses balance and falls, failing to complete the trajectory.

Gait Characteristics

Across all stable velocity tests (0.25–1.0 m/s), the robot demonstrates a consistent **trot** gait, characterized by diagonal foot-pair coordination and alternating stance-swing phases. This emergent gait arises naturally from the reward shaping and training dynamics without explicit gait supervision. The gait behavior was visualized using foot-ground contact plots, shown in Figs. 3.5-3.8.

At lower speeds (0.25 and 0.5 m/s), the gait exhibits longer stance durations and less aggressive leg movement. As the command velocity increases, the gait frequency rises and the robot becomes more dynamic while still retaining stability at 1.0 m/s, the maximum velocity seen during training.

Failure at High Speed Commands

At 2.0 m/s (Fig. 3.8), the robot fails to maintain stability and falls shortly after starting. This behavior highlights a key limitation: **when the commanded velocity exceeds the training**, the policy struggles to generalize.

If one wants to extend the maximum speed reachable by the robot, it is evident that a training with a broader set of velocities is needed. When dealing with wider ranges of commands and higher speeds, we can decide to train the policy gradually by introducing a curriculum, which allows the robot to begin training at easier levels and then progress to harder ones.

3.4 Simulation with added curriculum

In this section, we present the results obtained by a robot trained to walk on flat terrain at a speed uniformly varying between a fixed minimum velocity of 0.5 m/s and a variable "curriculum speed", which increases progressively from 0.5 m/s up to 2 m/s. The curriculum strategy used during training is defined by the following expressions:

```
self.curriculum_speed = min  $\left( \text{self.min\_vel} + \frac{\text{self.episode\_count}}{3000}, \text{self.max\_vel} \right)$ 
self._commands[env_ids] = torch.zeros_like(self._commands[env_ids])
                           .uniform(self.min_vel, self.curriculum_speed)
```

The following diagrams illustrate: (i) the robot's root link velocity compared to the commanded

velocity; (ii) the foot-ground contact patterns of each leg (RH, LH, RF, LF) over time.

Velocity Tracking

Fig. 3.9 shows how, by extending the range of velocities and introducing a curriculum in training, the robot is still able to track velocity commands.

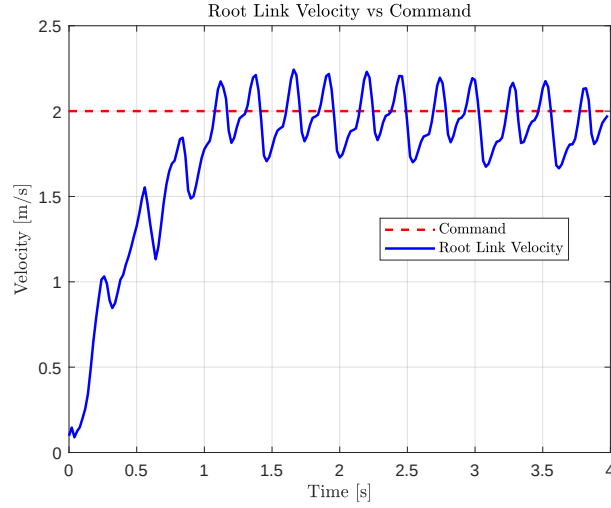


Figure 3.9: Velocity tracking analysis.

Trot-Run Gait Emergence

Fig. 3.10 demonstrates the emergence of a trot-run gait, despite the absence of an explicit gait rewarding in the reward design. A slight asymmetry in the (RF-LH) coordination/engagement is present but the robot still manages to keep a well-balanced rhythm.

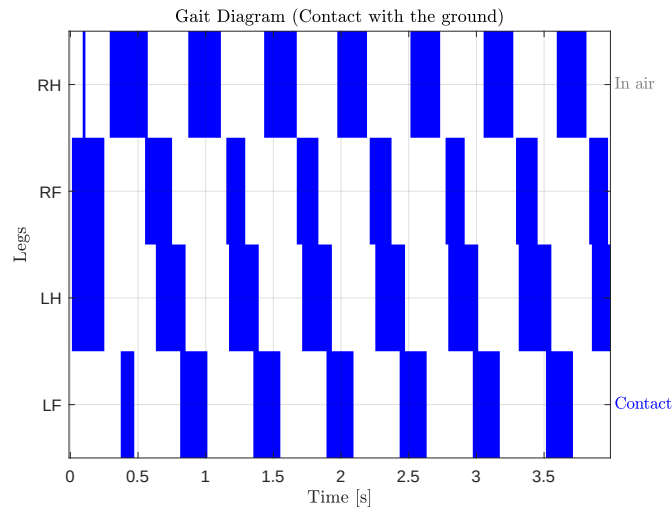


Figure 3.10: Gait analysis with a curriculum.

Chapter 4

Inducing a Bounding Gait

As is often the case, scientists stem their inspiration from observing nature. During the development of this project, we had the opportunity to watch a video of our colleague’s dog, Kumba, pictured in Fig. 4.1. At home, Kumba is a generally calm (lazy) and laid-back companion, who likes to rest during the days. However, once outdoors, he may become extremely lively and display bursts of energy when left to leap across open fields. Observing this behavior in the video file `kumba_running.mp4` made us ask whether we could teach a similar skill to our simulated robotic dog, ANYmal-C.

The previous locomotion policy, trained with a generic reward on flat terrain and variable command speeds, led to the spontaneous emergence of a trot-like gait — the most energy-efficient and dynamically stable pattern in quadrupeds at moderate speeds. However, to explore the possibility of enforcing alternative gait structures, we modified the reward function to bias the policy toward learning a **bound gait**, where both front legs (LF, RF) move in phase and alternate with both hind legs (LH, RH).



Figure 4.1: Kumba, an apparently innocuous (calm) dog.

The journey of teaching a custom gait to ANYmal-C took us three days and thirteen experiments. In detail, we attempted two very long, failing, overnight trainings, and a full day of profitable results and fine tuning. It is to say that the majority of the results discussed in this chapter come from the last day of training. Considering the ambitiousness of the task - learning a highly dynamic motion - and the fact that it was our very first trial of teaching a specific behavior to a simulated robot in IsaacLab, we felt very satisfied. We will be delighted to discuss the single steps of the process in the presentation attached to the report. In the remainder of the chapter we will summarize the main results and the necessary additions and modifications we performed on the RL environment.

4.1 Reward Shaping for Bounding Behavior

To induce bounding behavior, several terms were added to the default reward function, summarized in the following list. The rewards can be classified into two groups: the first is the main term - which stimulates the bound gait - and includes rewards from 1 to 4. They are combined and weighted by term 5. Terms from 6 to 9 represent fine-tuning terms and were added to correct drift of the robot and improve aesthetics of the motion.

1. **Front-leg Synchronization Reward:** A reward based on the squared difference between the contact states of LF and RF, encouraging them to be either simultaneously in contact with the ground or both in flight:

$$r_{\text{sync_front}} = \exp(-(\text{contact}_{\text{LF}} - \text{contact}_{\text{RF}})^2)$$

2. **Hind-leg Synchronization Reward:** Same as above but for LH and RH, promoting symmetrical hind-leg movement:

$$r_{\text{sync_hind}} = \exp(-(\text{contact}_{\text{LH}} - \text{contact}_{\text{RH}})^2)$$

3. **Front-Hind Alternation Reward:** To promote proper phasing between front and hind leg pairs, we compute the average contact of each pair and reward configurations where only one pair is in contact at a time:

$$r_{\text{desync}} = \exp(-(\text{avg}_{\text{front}} + \text{avg}_{\text{hind}} - 1)^2)$$

where $\text{avg}_{\text{front}} = (\text{contact}_{\text{LF}} + \text{contact}_{\text{RF}})/2$ and avg_{hind} is similarly defined. This encourages the bounding alternation pattern.

4. **Flight Phase Reward:** Bounding involves short aerial phases. A binary reward is given when no foot is in contact:

$$r_{\text{flight}} = \left[\sum_{\text{feet}} \text{contact} = 0 \right]$$

where $[\cdot]$ is the indicator function that is 1 when the condition inside is true, and 0 otherwise.

5. **Velocity-weighted Activation:** The previous reward terms are combined and weighted based on the coefficient w_{speed} , that increases from 0 to 1 as the commanded speed approaches 2 m/s:

$$w_{\text{speed}} = \text{clip} \left(\frac{\|v_{\text{cmd}}\|}{2.0}, 0, 1 \right)$$

This ensures bounding is only encouraged at dynamically suitable velocities.

6. **Hind-leg Joint Symmetry Reward:** Encourages symmetric motion between LH and RH joint configurations:

$$r_{\text{sym_hind}} = \exp(-\|\theta_{\text{LH}} - \theta_{\text{RH}}\|)$$

7. **Front-leg Joint Symmetry Reward:** Similar symmetry term for LF and RF joints:

$$r_{\text{sym_front}} = \exp(-\|\theta_{\text{LF}} - \theta_{\text{RF}}\|)$$

8. **Hip Abduction Penalty:** Penalizes excessive lateral hip (HAA) joint movement to encourage sagittal-plane bounding:

$$p_{\text{haa}} = \sum_{j \in \text{HAA}} \theta_j^2$$

9. **Speed Tracking Reward Override:** The linear velocity error reward was redefined by projecting the base velocity onto the commanded direction, leading to more accurate directional speed tracking:

$$r_{\text{vel}} = \exp\left(-\frac{(\|v_{\text{cmd}}\| - v_{\text{proj}})^2}{0.25}\right) \quad \text{where} \quad v_{\text{proj}} = \langle \hat{v}_{\text{cmd}}, v_{\text{actual}} \rangle$$

4.2 Simulations and Results

In order to train the robot to proceed along the desired direction the same curriculum used for the previous simulations was used, the only thing changing being the minimum velocity (now at 1.5 m/s) and the maximum velocity (now set at 3.0 m/s).

Following the modified rewards, the robot successfully learned a bounding gait at higher speeds. Below we report the same set of analysis plots for this new behavior, extracted at a commanded speed of 3.0 m/s.

Gait Pattern and Foot Contacts

In Fig. 4.2 a clear in-phase movement of front and hind legs can be observed, separated by distinct flight phases. This is consistent with a dynamically stable bound gait.

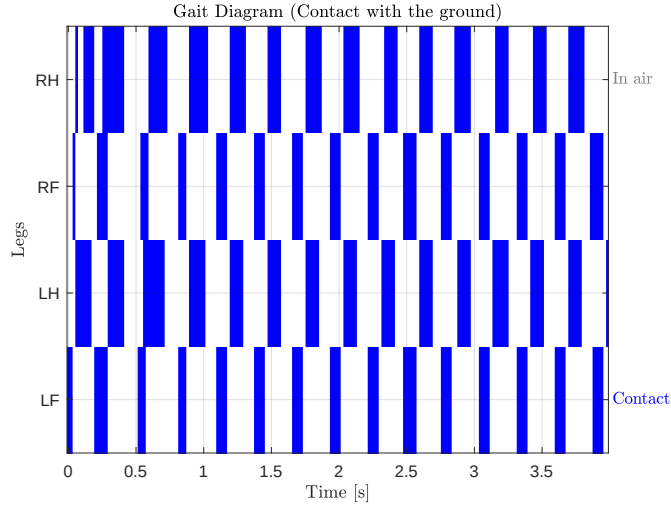


Figure 4.2: Bound Gait Feet Diagram.

Velocity Tracking

From a velocity tracking perspective, Fig. 4.3 shows proof of how a trade-off phenomenon must be taken into account when rewarding a certain behaviour. In fact, having tuned the rewards such that the resulting gait would have been a bound gait the reward for velocity tracking was slightly set aside. It can be noticed how the robot did not fullfill exactly the velocity tracking, though this must not be considered a failure as the real objective of this chapter was to prove that explicitly teaching a different gait is possible.

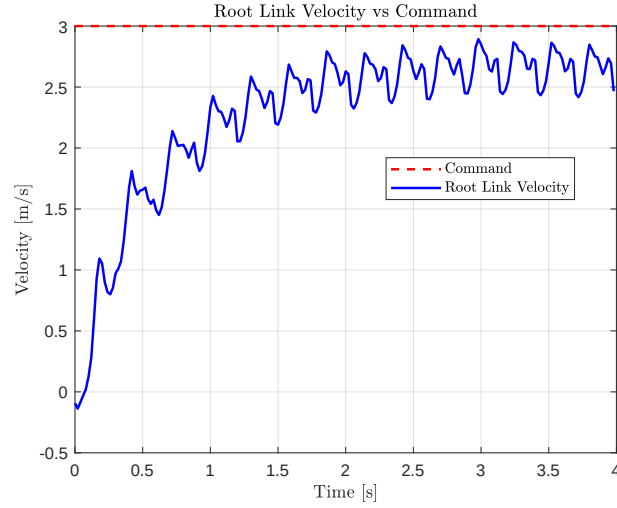


Figure 4.3: Velocity Tracking with Bound Gait Simulation

Use of Legs

Fig. 4.4 shows how each pair of legs' corresponding foot came in contact with the ground for roughly the same amount of time. It can be noticed that the hind pair of legs were in contact with the ground for longer than the front legs. This is because the bound gait inherently relies on a thrust phase provided by the hind legs, which are responsible for the push. The roughly equal amount of "use" between the left and right legs (independently of whether we consider the front or hind pair) confirms that the robot's motion, while executing a bound gait, is symmetric, and the rewards added to encourage better synchronization of the legs worked well.

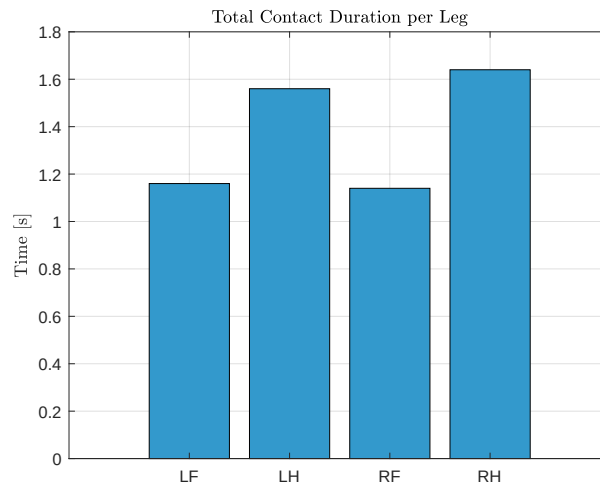


Figure 4.4: Total time each leg's corresponding foot touched the ground.

Autonomy Analysis

To estimate the autonomy of the robot, we computed the mechanical power consumption over time for each of the 12 joints. The instantaneous mechanical power was calculated as the product of applied joint torque and joint velocity. We took the absolute value of the power at each time step to avoid cancellation effects, and summed across all joints.

The total mechanical power was integrated over time by summing the discrete values and multiplying by the simulation time step ($\Delta t = 0.02$ s). This yielded the total mechanical energy consumed during the simulation. The average power was then calculated by dividing this energy by the total duration of the simulation. The average power computed is the mechanical one, applied on the torques. Accounting for drivetrain and system losses, the power that the battery must provide will be greater.

To estimate the autonomy, we considered a battery capacity of 932 Wh (retrieved from the ANYmal technical specifications [3]) and an overall efficiency factor $\eta = 0.64$. The overall efficiency factor was computed assuming an efficiency value of:

- 0.95 for motor drivers;
- an average of 0.85 for electric motors;
- 0.8 for reduction gears;

and then multiplying each of the single components' efficiency factors. The resulting autonomy was computed as:

$$\text{Autonomy (hours)} = \frac{932 \text{ Wh} \cdot \eta}{\text{Average Power (W)}}$$

Substituting the computed average power, the estimated autonomy is approximately 0.70 hours (about 42 minutes of the robot constantly moving with a bound gait at the velocity presented in the velocity tracking subsection).

Chapter 5

Conclusions

This project set out to explore IsaacLab’s simulation framework and RL techniques for control. The journey began by reading paper [4]. From there, we referred to a previous work, [1]. We learned about the limitations of model-based control strategies, particularly the challenge of accurately modeling actuator dynamics.

After collecting a broader knowledge on the topic, we installed the IsaacLab framework and followed the available tutorials [5]. This was a crucial part of the project, in that we were able to test the software functionalities before delving into our own project. We discovered that IsaacSim could be run even on hardware below the recommended requirements, by using strategic simplifications (e.g., `--cpu` flag) and smart memory allocations.

After achieving a sufficient confidence with the software’s capabilities, we focused on integrating the actuator network. Despite efforts to replicate the original authors’ results, we encountered misbehavior on the robot. Consequently, we reverted to the LSTM-based actuator model provided by IsaacLab.

A major milestone was the successful training of ANYmal-C on a biologically inspired bounding gait, using reinforcement learning techniques within the IsaacLab simulation framework. We carefully designed a custom reward function targeting front–hind synchronization, inter-limb phasing, and flight phases. After a series of iterations involving reward tuning and observation-driven corrections, the robot evolved from standing still to executing coordinated bounding motion across varying speeds.

Future Work

Although the project has successfully achieved a biologically inspired bound gait in simulation, several directions remain open for future investigation:

- **Improved Actuator Modeling:** Revisit the actuator network training pipeline using more general datasets (e.g., for all 12 joints) to deploy fully customized networks.
- **Alternative Gaits:** Extend the reward shaping and curriculum to teach other gaits, such as gallop and pacing, enabling gait transitions based on velocity commands.
- **Smooth gait transitions:** Investigate strategies for smoothly transitioning from a standing position to acceleration into a bound gait, and eventually decelerating into another gait (e.g., trot).
- **Domain Randomization and Sim-to-Real Transfer:** Introduce noise and varied terrain in simulation to train policies more robust to real-world uncertainties, enabling deployment

on physical ANYmal units (for example, the one belonging to PRISMA Lab).

Author Contribution

W.N analysed the Serial Elastic Actuators, investigated material provided by [1] and developed the training pipeline for learning ActNet. S.P performed analysis of the data logged from the simulation framework and provided plots for policy evaluation. Both authors helped setting up the hardware configuration for running the simulation framework. Both authors contributed to the reward shaping for bounding behavior and curriculum design. Both authors prepared the manuscript and presentation. W.N conceived the idea of teaching a bounding gait to ANYmal-C. S.P conceived the idea of analyzing battery autonomy. Every other unmentioned aspect of the project has been covered jointly by both the authors.

Acknowledgements

We thank **Davide Ruggiero** from STMicroelectronics for his seminar about AI and for providing useful material to train a neural network. We thank **Nudo's family** for their availability at sharing Kumba's photos and videos. We also thank **Prof. Fabio Ruggiero** for giving us the freedom of choice of the project's topic, allowing us to delve into state-of-the-art frameworks and concepts. Special thanks to **ChatGPT** and **Gemini** LLMs for supporting us in the development of the project and in writing the final report. But the biggest acknowledgments have to go to Scott McTominay and Ennio Morricone for keeping our morale high throughout all failures, and later successes, this project gifted us.

Bibliography

- [1] Jemin Hwangbo et al. “Learning agile and dynamic motor skills for legged robots”. In: *Science Robotics* 4.26 (Jan. 2019). ISSN: 2470-9476. DOI: 10.1126/scirobotics.aau5872. URL: <http://dx.doi.org/10.1126/scirobotics.aau5872>.
- [2] *Supplementary materials for "Learning Agile and Dynamic Motor Skills for Legged Robots"*. URL: https://github.com/junja94/anymal_science_robotics_supplementary.
- [3] *ANYmal Technical Specifications*. URL: <https://www.anybotics.com/anymal-technical-specifications.pdf>.
- [4] Nikita Rudin et al. *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning*. 2022. arXiv: 2109.11978 [cs.R0]. URL: <https://arxiv.org/abs/2109.11978>.
- [5] *Isaac Lab Documentation*. URL: <https://isaac-sim.github.io/IsaacLab/main/index.html>.